Soft-DTW: a Differentiable Loss Function for Time-Series

Reproduction of Cuturi & Blondel (ICML 2017) and an extension: soft-DTW with a warping penalty

Pablo Borrego Pol Tibau

MVA – Machine Learning for Time Series

December 2026

Comparing series that are shifted or stretched in time

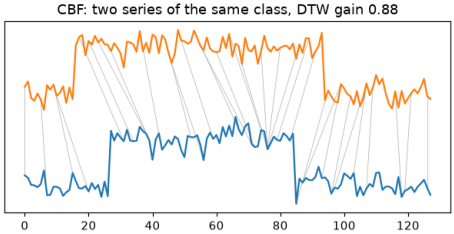
DTW: the cheapest monotone alignment

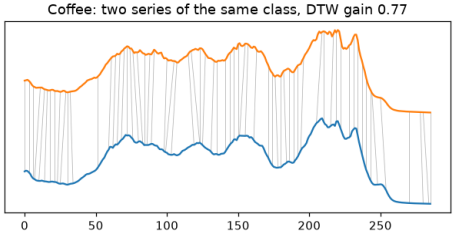
$$\mathrm{dtw}(x, y) = \min_{A \in \mathcal{A}_{n,m}} \langle A, \Delta(x, y) \rangle$$

with costs $\delta_{i,j} = \|x_i - y_j\|^2$; computed by dynamic programming in $O(nm)$.

DTW it is not differentiable

The minimum jumps from one path to another when x changes: DTW has kinks and cannot be used directly as a training loss.





Soft-DTW replaces the minimum by a soft-minimum

$$\min^\gamma(a_1, \dots, a_k) = -\gamma \log \sum_I e^{-a_I/\gamma}$$

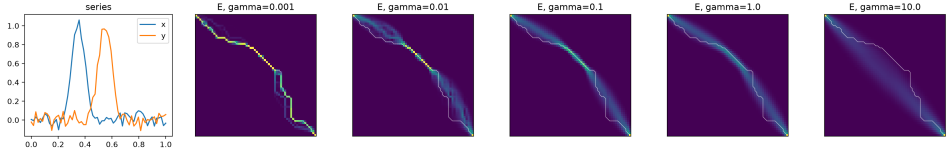
$$\mathrm{dtw}_\gamma(x, y) = \min_{A \in \mathcal{A}_{n,m}}{}^{\gamma} \langle A, \Delta(x, y) \rangle$$

- $\gamma \to 0$ recovers DTW
- $\gamma > 0$: smooth in x

Its gradient has a meaning

$$\nabla_x \mathrm{dtw}_\gamma = \sum_{i,j} e_{i,j} \frac{\partial \delta_{i,j}}{\partial x}, \qquad E = \mathbb{E}_\gamma[A]$$

$e_{i,j}$: share of all alignments that match x_i with y_j, each alignment A counting with weight $e^{-\mathrm{cost}_A/\gamma}$.



E for increasing γ: the DTW path (white) becomes a diffuse band.

Two $O(nm)$ recursions, implemented from scratch

Forward (Alg. 1)

Bellman's recursion: $r_{i,j} = \delta_{i,j} + \min^{\gamma}$ of its 3 neighbours $r_{i-1,j-1}$, $r_{i-1,j}$, $r_{i,j-1}$. Value: $r_{n,m}$.

Backward (Alg. 2)

From (n, m) back to $(1, 1)$, each cell collects $e_{i,j}$ from its 3 successors: E and the gradient.

Gradient

Chain rule through the costs: $\nabla_{x_i} \mathrm{dtw}_{\gamma} = \sum_j e_{i,j}\, 2(x_i - y_j)$. Same $O(nm)$ cost as the value.

Our implementation

Numba core

Both recursions compiled with Numba, soft-min computed stably (log-sum-exp). 0.3 ms for $n = m = 64$.

PyTorch loss

Usable in any network, with a hand-written backward: autograd through the recursion is 2500× slower.

Verified

For $n, m \leq 5$, listing every alignment gives the same value and E (error $< 10^{-10}$); gradients match finite differences.

Data

The datasets

- 13 datasets of the UCR archive (the paper used 79)
- ECG, motion, shapes, power demand, synthetic
- Lengths 24 to 512; 2 to 60 classes

Do they need DTW?

Question: are two series of the same class misaligned in time?

Test: compare their distance before and after aligning them:

$$\text{DTW gain} = 1 - \frac{\mathrm{dtw}(x, y)}{\|x - y\|^2}$$

= share of the distance that disappears once aligned.

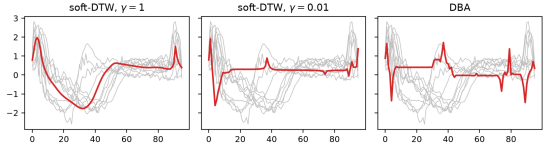
Answer: 48–96% on every dataset, so warping matters for all of them.

Barycenters and k-means

% datasets where soft-DTW beats DBA	$\gamma = 1$	$\gamma = 0.1$	$\gamma = 0.01$	$\gamma = 0.001$
Barycenters, random init	58 (41)	92 (94)	100 (100)	100 (97)
Barycenters, Euclidean-mean init	8 (4)	25 (47)	75 (80)	100 (90)
k-means, random init	10 (16)	50 (25)	70 (60)	90 (77)
k-means, Euclidean init	10 (29)	10 (24)	60 (55)	90 (69)

Ours (12 datasets for barycenters, 10 for k-means) vs the paper (79) in parentheses. All scored with the hard DTW loss.

- From a random start, the soft-DTW barycenter has a **14% lower** DTW loss than DBA's (74% lower on Plane).
- From the Euclidean mean, it is only **3% lower**.
- With $\gamma = 0.001$ it takes **14 times** as long as DBA to compute.

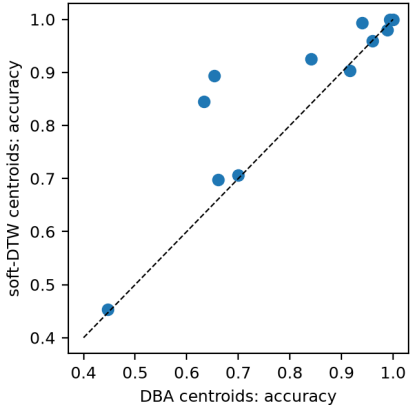


Nearest centroid

	Ours	Paper
% datasets where soft-DTW $\geq$ DBA	83	75
Mean accuracy, DBA	0.81	–
Mean accuracy, soft-DTW	0.86	–

Ours: 12 datasets, 3 random splits; paper: 79 datasets.

- Each class is represented by the barycenter of its training series; a new series gets the class of the **closest centroid**.
- γ is chosen on a validation split.
- Soft-DTW centroids classify **at least as well** as DBA's on 83% of the datasets: better barycenters represent the class better.



One point per dataset; above the diagonal, soft-DTW wins.

Forecasting

An MLP sees the first 60% of a series and predicts the rest, trained with the Euclidean loss or soft-DTW, from scratch or from the Euclidean network. Average rank over 13 datasets (1 = best; paper in parentheses):

Test metric, init	Euclidean	$\gamma = 1$	$\gamma = 0.1$	$\gamma = 0.01$	$\gamma = 0.001$
DTW, from scratch	3.69 (3.46)	3.92 (3.55)	2.31 (3.33)	2.23 (2.79)	2.85 (1.87)
DTW, from Euclidean	3.92 (4.21)	4.46 (3.96)	2.85 (3.42)	2.08 (2.12)	1.69 (1.29)
Euclidean, from scratch	1.31 (1.05)	1.69 (2.41)	3.46 (3.42)	4.00 (4.13)	4.54 (3.99)

What the table shows

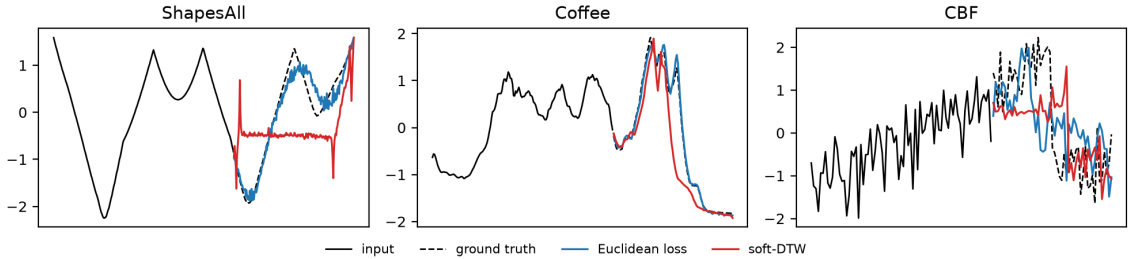
Each loss wins on its own metric: soft-DTW with a small γ ranks best in DTW, the Euclidean loss in Euclidean distance. From the Euclidean network, the smaller γ, the better, as in the paper.

Outliers

From scratch, $\gamma = 0.001$ is unstable: on Coffee the test DTW is $3.2\times$ the Euclidean network's. On ShapesAll the forecast degenerates (next slide).

A degenerate solution

On ShapesAll, soft-DTW predicts a flat line with spikes, and DTW scores it better than the Euclidean forecast (6.6 vs 9.3).



Why

DTW can match one point of the prediction with a whole segment of the truth: **stretching time is free**, so one spike at the right height "covers" an entire rise.

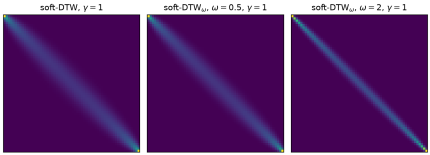
Soft-DTW$_\omega$: a penalty ω for every stretching step

$$r_{i,j} = \min{}^{\gamma}\{r_{i-1,j-1}+\delta_{i,j},\ r_{i-1,j}+\delta_{i,j}+\omega,\ r_{i,j-1}+\delta_{i,j}+\omega\}$$

- Cost of an alignment: $\langle A, \Delta \rangle + \omega \times (\#\text{stretching steps})$
- Same $O(nm)$ forward and backward (soft DP with step costs, Mensch & Blondel 2018)
- $\partial/\partial\omega$ = expected number of stretching steps

Limit	Recovers
$\omega = 0$	soft-DTW
$\omega \to \infty$	Euclidean distance
$\gamma \to 0$	amerced DTW

Amerced DTW: Herrmann & Webb, 2023.



E with zero costs: the larger ω, the less time is stretched.

The penalty helps when the problem is getting lost

Experiment	Soft-DTW$_\omega$ vs soft-DTW (same γ, same start)
Barycenters, random start	8% lower DTW loss, 1.3× faster
Barycenters, Euclidean-mean start	Worse: DTW loss 7% ($\omega = 0.1$) to 30% ($\omega = 1$) higher. As $\gamma \to 0$ it tends to DTW + ω, not to the DTW loss used for scoring
k-means	Worse: DTW loss 10% ($\omega = 0.1$) to 50% ($\omega = 1$) higher
Nearest centroid	No gain: 0.85 vs 0.86 mean accuracy
Forecasting, from scratch	$\omega = 0.1$: same DTW, 30% lower Euclidean error. $\omega = 1$: Euclidean error ÷ 2 to 3 on all 13 datasets

ω pays off from a poor starting point (random weights, random barycenters), where free warping lets the optimisation fall into degenerate alignments. From a good start, plain soft-DTW is closer to what is measured.

Conclusion

Reproduced

Exact implementation (brute force, finite differences). The paper's four claims hold on our 10–13 datasets.

Nuanced

Large gains only from poor starts; 14–16$\times$ DBA's cost at $\gamma = 0.001$; unstable, degenerate forecasts from scratch.

Extended

Soft-DTW$_\omega$ fixes the forecasting failures and trades shape for timing; it does not help from a good start.

Limitations. ω is set by hand (minimising the loss itself drives $\omega \to 0$, so it must be tuned on another objective) and depends on the scale of the costs; 13 datasets instead of 79.

Thank you. Questions?